

Implementation

4.1 System Overview

The implemented system is a mobile-first, AI-powered dermatology assistant capable of classifying three common skin conditions — Acne, Eczema, and Tinea — from clinical photographs, while providing visual explanations that indicate which regions of the image drove the prediction. The system integrates three interconnected components: a multi-phase deep learning pipeline trained on a curated dermatological dataset, a three-stage on-device inference engine, and a cross-platform Flutter mobile application.

The end-to-end workflow follows this sequence: the user captures or uploads a skin image through the application; the image is passed through a Variational Autoencoder (VAE) that acts as an anomaly detection gate to confirm the presence of a skin lesion; if a lesion is detected, the image is fed into an ensemble of two EfficientNet classifiers (EfficientNetB2 and EfficientNetB3) that produce a three-class probability distribution; finally, a Score-CAM algorithm generates a heatmap overlay highlighting the lesion regions most relevant to the classification decision. All inference runs entirely on-device using TensorFlow Lite, requiring no network connectivity after installation.

4.2 Development Environment and Tools

The project utilized two distinct environments: a local Windows 10 workstation for application development and initial prototyping, and Kaggle cloud GPU notebooks for computationally intensive model training.

4.2.1 Machine Learning Stack

Table 4.1 — Machine Learning Tools and Frameworks

Component	Tool / Version
Deep learning framework	TensorFlow / Keras (Python)
GPU training platform	Kaggle (NVIDIA P100 / T4)
Model architectures	MobileNetV2, EfficientNetB0–B3
Image processing (Python)	OpenCV (cv2), NumPy
Class balancing	scikit-learn (compute_class_weight)
TFLite conversion	tf.lite.TFLiteConverter

Random seed	42 (all experiments)
-------------	----------------------

4.2.2 Mobile Application Stack

Table 4.2 — Flutter Application Dependencies

Component	Tool / Version
Framework	Flutter 3.x
Language	Dart 3.x
State management	Riverpod (flutter_riverpod ^2.4.9)
Navigation	GoRouter (go_router ^13.0.0)
On-device AI inference	TFLite Flutter (tflite_flutter)
Image processing (Dart)	image package
Local persistence	SQLite (sqflite ^2.3.0)
Camera integration	camera ^0.10.5+5
Image picker	image_picker ^1.0.5
File paths	path_provider ^2.1.1
Runtime permissions	permission_handler ^11.1.0

4.3 Dataset Preparation

4.3.1 Raw Dataset and Cleaning

The initial dataset was organized into three class folders — Acne, Eczema, and Tinea — under a train/test directory structure sourced from publicly available dermatological image repositories. Before any model training, the dataset underwent systematic quality control. Duplicate images were identified and removed to prevent data leakage and artificially inflated performance metrics. Images that were clearly non-dermatological (histology slides, diagrams, or unrelated photographs), severely low-resolution, or heavily watermarked were excluded. Misclassified images identified through visual inspection were either relabeled or discarded. The cleaned output was stored in a dedicated directory, *Finalized_Clean_Data/*.

The original two-way (train/test) split was restructured into a three-way (train/validation/test) split with stratification to preserve class proportions across all three partitions. Approximately 90% of images were allocated to training, with 5% each for validation and test. The validation set was used exclusively for hyperparameter tuning and early stopping decisions; the test set was held out entirely and used only for final evaluation.

4.3.2 Offline Data Augmentation

Data augmentation was applied offline to the training split only, using a deterministic Keras preprocessing pipeline. Each original training image was transformed three times, producing three distinct augmented copies. The augmentation pipeline was implemented as follows:

```
augmentation_pipeline = tf.keras.Sequential([
    tf.keras.layers.RandomFlip(mode="horizontal", seed=42),
    tf.keras.layers.RandomFlip(mode="vertical", seed=42),
    tf.keras.layers.RandomRotation(factor=0.0417, # ±15 degrees
    fill_mode="reflect", seed=42),
    tf.keras.layers.RandomZoom(height_factor=0.1, width_factor=0.1,
    fill_mode="reflect", seed=42),
    tf.keras.layers.RandomBrightness(factor=0.1, seed=42),
])
```

Table 4.3 — Data Augmentation Techniques

Technique	Parameters	Rationale
Horizontal Flip	50% probability	Skin lesions have no left-right orientation
Vertical Flip	50% probability	Augments viewpoint diversity
Random Rotation	±15 degrees, reflect fill	Simulates camera angle variation
Random Zoom	±10%, reflect fill	Mimics varying capture distances
Random Brightness	±10%	Accounts for lighting variation

The choice of `fill_mode="reflect"` avoided introducing black border artifacts that can be misinterpreted as disease features, and a fixed random seed of 42 ensured full reproducibility. Augmented images were saved with the naming convention `aug_{original_name}_{index}.jpg`. The validation and test sets were intentionally left unmodified to ensure unbiased evaluation.

4.3.3 Final Dataset Statistics

The augmented dataset, stored in *New_Augmented_Dataset/*, has the following composition after augmentation was applied exclusively to the training split:

Table 4.4 — Final Dataset Distribution Across Splits and Classes

Split	Acne	Eczema	Tinea	Total
Training	2,880	3,460	1,816	8,156
Validation	93	142	90	325
Test	96	137	89	322

Total	3,069	3,739	1,995	8,803
--------------	--------------	--------------	--------------	--------------

A notable class imbalance exists in the training set: Eczema comprises approximately 42.4% of training samples, while Tinea accounts for only 22.3%. This imbalance was addressed throughout training using dynamically computed class weights.

4.4 Model Development

4.4.1 Training Configuration and Data Pipeline

All training pipelines followed a consistent data loading and optimization pattern. Images were loaded using `tf.keras.utils.image_dataset_from_directory` with categorical labels, then processed through a pipeline combining in-memory caching and prefetching:

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.map(lambda x, y: (preprocess_input(x), y),
num_parallel_calls=AUTOTUNE)
train_ds = train_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

The `.cache()` call stored the preprocessed dataset in memory after the first epoch, eliminating redundant I/O and preprocessing in subsequent epochs. The `.prefetch(AUTOTUNE)` operation overlapped CPU data preparation with GPU computation, maximizing GPU utilization during training on Kaggle.

To address class imbalance, dynamic class weights were computed using scikit-learn before each training run, assigning weights inversely proportional to class frequency: $w_c = N / (k \times n_c)$, where N is total samples, k is number of classes, and n_c is the count for class c . Tinea, as the minority class, received the highest weight throughout all training phases.

4.4.2 Phase 1: MobileNetV2 Transfer Learning

The first training phase established a baseline using MobileNetV2 pretrained on ImageNet as a frozen feature extractor. MobileNetV2 was selected for its lightweight depthwise separable convolution architecture, which offered a strong balance between classification accuracy and mobile inference efficiency.

The entire MobileNetV2 backbone was frozen (`base_model.trainable = False`), and a lightweight classification head was appended:

```

base_model = MobileNetV2(weights="imagenet", include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = False

inputs = tf.keras.Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(3, activation="softmax")(x)
model = models.Model(inputs, outputs)

```

The model was compiled with Adam (learning rate = 1×10^{-4}) and categorical cross-entropy loss, trained for up to 20 epochs with EarlyStopping (patience = 3, monitoring validation loss). Images were preprocessed using MobileNetV2's native `preprocess_input` function, which scales pixel values from $[0, 255]$ to $[-1, 1]$.

This phase achieved **75.16% test accuracy**, with strong performance on Acne (80.21%) and Eczema (81.75%), but weaker on Tinea (59.55%) due to the frozen backbone's inability to adapt to dermatology-specific features.

Table 4.5 — Phase 1 (MobileNetV2 Transfer Learning) Configuration

Parameter	Value
Input size	$224 \times 224 \times 3$
Preprocessing	<code>preprocess_input</code> (scales to $[-1, 1]$)
Batch size	32
Optimizer	Adam (learning rate = 1×10^{-4})
Loss function	Categorical Cross-Entropy
Classification head	GAP $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(3, softmax)
Callback	EarlyStopping (patience=3, restore best weights)
Test accuracy	75.16%

4.4.3 Phase 2: Fine-Tuning (Layer Unfreezing)

Phase 2 loaded the Phase 1 model and selectively unfroze the upper layers of the MobileNetV2 backbone to allow domain adaptation. The first 100 of MobileNetV2's 155 layers — which encode low-level features such as edges and textures — remained frozen, while layers 100 onward were made trainable:

```
base_model.trainable = True
for i, layer in enumerate(base_model.layers):
    layer.trainable = (i >= 100)
```

A drastically reduced learning rate of 1×10^{-4} — one tenth of Phase 1's rate — was critical to prevent catastrophic forgetting of the pretrained representations. Three callbacks were employed: **EarlyStopping** (patience=5, increased from 3 to accommodate the slower convergence of fine-tuning), **ReduceLROnPlateau** (factor=0.5, patience=2, min_lr= 1×10^{-4}), and **ModelCheckpoint** (saving only the epoch with the best validation loss).

This phase produced the most significant single improvement across all MobileNetV2 experiments: **84.16% test accuracy**, an improvement of 8.99 percentage points over Phase 1. Tinea recall improved from 59.55% to 71.91%, and Eczema recall reached 91.24%. The model was saved as `mobilenetv2_finetuned_model.keras` (23.45 MB).

Table 4.6 — Phase 2 Per-Class Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
Acne	0.9213	0.8542	0.8865	96
Eczema	0.8065	0.9124	0.8562	137
Tinea	0.8205	0.7191	0.7665	89
Macro avg	0.8494	0.8286	0.8364	322
Weighted avg	0.8446	0.8416	0.8404	322

4.4.4 Phase 3: Anti-Overfitting Strategies

Phase 3 explored a comprehensive suite of regularization techniques to further improve generalization. Five complementary strategies were implemented simultaneously:

- **Conservative Fine-Tuning:** Only the top 50 layers of the MobileNetV2 backbone were made trainable (compared to 55 in Phase 2), reducing the number of free parameters.
- **Enhanced Regularization:** Dropout was increased from 0.2 to 0.5; L2 weight decay of 0.001 was added; gradient clipping (clipnorm=1.0) prevented exploding gradients.
- **Label Smoothing:** A smoothing factor of 0.2 was applied to the categorical cross-entropy loss, replacing hard one-hot targets with soft distributions to discourage overconfidence and improve calibration.
- **Online MixUp Augmentation:** Synthetic samples were generated by linearly interpolating between image pairs and their labels ($\lambda \sim \text{Uniform}(0, 0.2)$), encouraging smoother decision boundaries.
- **Warmup Cosine Decay Schedule:** A custom learning rate callback implemented a 5-epoch linear warmup from 1×10^{-4} to 1×10^{-3} , followed by cosine annealing for the remaining

epochs.

Despite the sophistication of these techniques, Phase 3 recorded **73.91% test accuracy** without Test Time Augmentation (TTA) and 76.09% with TTA — below Phase 2's 84.16%. The under-performance stemmed from restarting fine-tuning from the Phase 1 base model (less-adapted weights) combined with an excessively low learning rate (1×10^{-3}) and strong regularization, which together prevented convergence to an equally good minimum. This result confirmed that Phase 2's configuration — 55 unfrozen layers, learning rate 1×10^{-3} , moderate dropout — represented the optimal trade-off for this dataset size (~8,000 training images).

4.4.5 Transition to EfficientNet

Following the MobileNetV2 experiments, the architecture was upgraded to the EfficientNet family to push accuracy further. EfficientNet models are designed through compound scaling — simultaneously scaling network depth, width, and input resolution using a principled coefficient — resulting in significantly higher accuracy at equivalent parameter budgets.

Two key design decisions distinguished the EfficientNet training from the earlier MobileNetV2 experiments:

- **Focal Loss:** Standard categorical cross-entropy was replaced with `CategoricalFocalCrossentropy(alpha= $\alpha$ , gamma=2.0)`, where alpha is per-class inverse-frequency weighting. Focal loss down-weights well-classified examples and focuses training on hard, misclassified samples — particularly beneficial for the Tinea class.
- **Online Data Augmentation Within the Model Graph:** Augmentation layers (RandomFlip, RandomRotation, RandomZoom, RandomTranslation, RandomContrast, RandomBrightness) were embedded directly as the first layers of the model, applied stochastically during each training pass and automatically bypassed at inference time (`training=False`).

4.4.6 EfficientNetB2 Training

EfficientNetB2 uses a native input resolution of 260×260 pixels and has approximately 339 layers. Training followed a two-phase approach executed entirely on Kaggle GPU:

Phase 1 — Frozen Backbone: The backbone was entirely frozen and only the classification head was trained using Adam ($\text{lr} = 1 \times 10^{-3}$) with focal loss. The classification head included a 256-unit Dense layer with BatchNormalization and Dropout(0.45) for additional regularization, followed by the 3-class softmax output.

Phase 2 — Selective Unfreezing: The best Phase 1 checkpoint was loaded and the top ~120 layers (from layer 220 onward) of the backbone were unfrozen. The model was recompiled with $\text{lr} = 1 \times 10^{-3}$ and trained for up to 60 epochs. An Eczema weight boost factor of $1.2 \times$ was applied on top of the balanced class weights, calibrated through experimentation to prevent the majority class from compressing per-class performance.

Table 4.7 — EfficientNetB2 Training Configuration

Parameter	Phase 1	Phase 2
Input resolution	260 × 260	260 × 260
Backbone	Frozen	Top ~120 layers unfrozen
Learning rate	1×10 ^{−3}	1×10 ^{−4}
Loss	Focal ($\gamma=2.0$)	Focal ($\gamma=2.0$)
Max epochs	20	60
EarlyStopping patience	15	15
ReduceLROnPlateau	factor=0.3, patience=8	factor=0.3, patience=8

4.4.7 EfficientNetB3 Training

EfficientNetB3 operates at 300×300 pixels and has approximately 385 layers, offering greater representational capacity than B2. The same two-phase training procedure was applied: frozen backbone training at $\text{lr} = 1 \times 10^{-3}$, followed by fine-tuning with the top 150 layers unfrozen (backbone layers 235 onward) at $\text{lr} = 1 \times 10^{-4}$, with up to 60 epochs and EarlyStopping (patience=15).

EfficientNetB3 serves a dual role in the final pipeline: as a classifier in the ensemble and as the backbone for Score-CAM feature extraction. Its `top_activation` layer produces feature maps of shape (1, 10, 10, 1536), which form the basis for the Score-CAM heatmap computation described in Section 4.5.3.

4.5 Three-Stage Inference Pipeline

The final deployed system implements a sequential three-stage inference pipeline. Each stage acts as a gate or enrichment step, adding progressively more specific information about the input image. The pipeline design ensures that the most computationally expensive operations (CNN classification and Score-CAM) are only invoked when a genuine skin lesion has been detected.

4.5.1 Stage 1: VAE Anomaly Detection Gate

The first stage uses a Variational Autoencoder (VAE) trained on normal skin images as an anomaly gate. Images that do not exhibit skin disease characteristics are flagged as normal and rejected before reaching the classifier, preventing spurious disease classifications on non-clinical photographs.

The VAE is applied using a sliding window approach over the input image. The image is first downsampled to a maximum width of 1,280 pixels (if larger) using linear interpolation. A 64×64-pixel patch is then extracted at each position with a stride of 32 pixels, producing

overlapping coverage of the entire image. Each patch is fed into the VAE, which reconstructs it; the mean squared error (MSE) between the original and reconstructed patch is the anomaly signal.

```
for (int y = 0; y <= h - patchSize; y += stride) {
  for (int x = 0; x <= w - patchSize; x += stride) {
    // Extract 64×64 patch
    final patch = img.copyCrop(original, x: x, y: y,
      width: patchSize, height: patchSize);
    _fillFloat32(patch, inputBuf, patchSize, patchSize); // normalize to
    [0,1]
    _vae!.invoke();
    final mse = _vae!.getOutputTensor(0).data
      .buffer.asFloat32List()[0];
    patches++;
    if (mse > anomalyThreshold) anomalous++;
  }
}
```

An MSE exceeding the calibrated threshold of **0.008** (adjusted from the original PyTorch threshold of 0.006 to account for a systematic TFLite quantization offset of ~0.002) marks a patch as anomalous. If the ratio of anomalous patches to total patches exceeds **20%**, the image proceeds to Stage 2; otherwise, the pipeline returns a "No skin disease detected" result immediately.

4.5.2 Stage 2: CNN Ensemble Classification

Images flagged as anomalous proceed to the classification stage, where an ensemble of two independently trained EfficientNet models produces the final diagnosis. Ensemble methods reduce prediction variance and improve generalization compared to any single model. The two models' different input resolutions (260×260 for B2 vs. 300×300 for B3) expose them to slightly different feature scales, making their predictions complementary.

Because both EfficientNet models include an internal Rescaling(1/255) layer baked into the backbone, the application passes raw pixel values in the range [0, 255]. Sending pre-normalized [0, 1] values would result in double-normalization and near-black inputs, causing the classifier to produce near-uniform probability distributions. This contrast with the VAE, which requires normalized [0, 1] input, was a critical distinction implemented through separate preprocessing helper functions.

```
// Ensemble: 50/50 average of B2 and B3 softmax outputs
List<double> ensemble = List.generate(
  3, (i) => (probsB2[i] + probsB3[i]) / 2.0);

final predIdx = _argmax(ensemble);
final diagnosis = _classes[predIdx]; // "Acne" | "Eczema" | "Tinea"
final confidence = ensemble[predIdx];
```

4.5.3 Stage 3: Score-CAM Explainability

The third stage generates a Score-CAM (Score-weighted Class Activation Map) heatmap that visually highlights which regions of the skin image most strongly influenced the classification decision. Unlike Grad-CAM, Score-CAM is gradient-free: it uses the classifier's forward-pass response to channel-masked images to assign importance scores. This makes it directly compatible with TFLite models, which do not support gradient computation on-device.

The algorithm proceeds through six steps:

- **Feature Extraction:** The image (resized to 300×300) is fed into the B3 feature extractor TFLite model, producing a tensor of shape (1, 10, 10, 1536) — 1,536 channels, each a 10×10 spatial activation map.
- **Top-K Channel Selection:** The 1,536 channels are ranked by mean absolute activation magnitude. The top K=30 channels are selected, reducing masked forward passes from 1,536 to 30 (~3 seconds on a mid-range device).
- **Masked Forward Passes:** For each selected channel, the 10×10 activation map is upsampled to 300×300 and normalized to [0, 1], then applied as a spatial mask: `masked_image = original_image × attention_mask`. The masked image is passed through the B3 classifier and the target class probability is recorded as the channel's importance score.
- **Score Normalization:** The 30 raw scores are passed through a numerically stable softmax to produce weights summing to 1.
- **Weighted Heatmap:** The final heatmap is the weighted sum of the 30 upsampled feature maps. A ReLU operation removes negative contributions, and the result is min-max normalized to [0, 1].
- **Visualization:** The heatmap is colored using a jet colormap (blue = low importance, red = high importance), resized to the original image dimensions, and blended with the original image at a 60%/40% ratio (original/heatmap). The overlay is saved as a JPEG to the device's temporary directory.

4.6 TFLite Model Conversion and Deployment

Four TFLite models are used at runtime and are bundled in the application's assets directory (assets/models/). Conversion was performed using `tf.lite.TFLiteConverter.from_keras_model()` with `converter.optimizations = [tf.lite.Optimize.DEFAULT]`, which applies dynamic range quantization to reduce model size and improve inference latency on mobile hardware.

Table 4.8 — TFLite Models Bundled in the Application

File	Purpose	Input Shape	Input Range
vae_model.tflite	VAE anomaly gate	(1, 64, 64, 3) float32	[0, 1]

cnn_b2_model.tflite	EfficientNetB2 classifier	(1, 260, 260, 3) float32	[0, 255]
cnn_b3_model.tflite	EfficientNetB3 classifier	(1, 300, 300, 3) float32	[0, 255]
b3_feature_extractor.tflite	Score-CAM feature maps	(1, 300, 300, 3) float32	[0, 255]

The training-time augmentation layers embedded in the EfficientNet model graph are automatically removed by the converter, as they are no-ops at inference time (`training=False`). Prior to deployment, each TFLite model was verified for determinism — two forward passes on the same input produced identical outputs (maximum absolute difference $< 1 \times 10^{-6}$) — ensuring consistent classification results on the device.

4.7 Flutter Mobile Application

4.7.1 Application Architecture

The Flutter application is structured following a feature-based directory organization, cleanly separating concerns across onboarding, core workflow, and results domains:

```
Flutter/lib/
  ■■■ main.dart # App entry, ProviderScope
  ■■■ core/
    ■ ■■■ routing/app_router.dart # GoRouter configuration
    ■ ■■■ theme/app_theme.dart # Light and dark theme definitions
    ■ ■■■ widgets/ # Shared UI components
    ■■■ features/
      ■ ■■■ onboarding/screens/ # Welcome, Login, Role, PersonalInfo,
      ■ ■ AllSet, HowItWorks, ForgotPassword
      ■ ■■■ core_workflow/screens/ # Home, BodyPartSelection, Capture,
      ■ ■ Analyzing, History, Profile,
      ■ ■ EditProfile, Notifications, HelpSupport
      ■ ■■■ results/screens/ # DiagnosisResult
    ■■■ models/
      ■ ■■■ scan_result.dart # Scan data model
      ■ ■■■ user.dart # User data model
    ■■■ providers/
      ■ ■■■ scan_provider.dart # Scan state management
      ■ ■■■ user_provider.dart # User/auth state management
    ■■■ services/
      ■■■ ai/ai_service.dart # Full inference pipeline (singleton)
      ■■■ database/database_service.dart # SQLite CRUD operations
```

4.7.2 State Management with Riverpod

The application uses Riverpod for all reactive state management. The root widget is wrapped in a `ProviderScope`, enabling compile-time safe access to providers throughout the widget tree. `DermatologyAIApp` is a `ConsumerWidget` that watches both a router provider and a theme mode provider, ensuring the application rebuilds correctly in response to either state change.

Riverpod was preferred over BLoC or the basic `Provider` package for its compile-time type safety, its independence from the widget tree (providers are accessible without requiring a `BuildContext`), and its clean handling of asynchronous inference states through `AsyncValue`.

4.7.3 Navigation with GoRouter

All application navigation is handled declaratively through `go_router`. Route paths are defined as typed constants in an `AppRoutes` class, preventing hard-coded string errors. The router handles the complete user journey in three phases:

- **Onboarding Flow:** Welcome → Login → Role Selection → Personal Info → All Set
- **Core Scanning Workflow:** Home Dashboard → Body Part Selection → Capture → Analyzing → Diagnosis Result
- **Auxiliary Screens:** Scan History, Profile, Edit Profile, Notifications, Help & Support, Specialist Dashboard (Healthcare Provider role)

Table 4.9 — Application Screen Routes

Route	Screen	Description
/welcome	WelcomeScreen	App introduction and entry point
/login	LoginScreen	User authentication
/role-selection	RoleSelectionScreen	Patient or Healthcare Provider
/personal-info	PersonalInfoScreen	Demographic data collection
/dashboard	HomeScreen	Main hub with quick actions
/body-part-selection	BodyPartSelectionScreen	Anatomical region selection
/capture	CaptureScreen	Camera or gallery image capture
/analyzing	AnalyzingScreen	Real-time inference progress
/diagnosis-result	DiagnosisResultScreen	Classification result with heatmap
/history	HistoryScreen	Chronological scan history

4.7.4 AI Service Integration

The `AIService` class is implemented as a singleton and provides the full three-stage inference pipeline as a single asynchronous entry point, `analyzeImage()`. It accepts an image file path and an optional progress callback that fires with a step index (0–3) as each stage begins, enabling the UI to display a real-time progress indicator during analysis:

```
Future<AnalysisResult> analyzeImage({
  required String imagePath,
  void Function(int step)? onStepChange, // 0=preprocess 1=VAE 2=CNN
  3=ScoreCAM
}) async { ... }
```

The four TFLite interpreters (VAE, B2, B3, and B3 feature extractor) are loaded lazily on first call via `Interpreter.fromAsset()` and held in memory for the application's lifetime, avoiding re-initialization overhead on each scan. To keep the UI responsive during the 30-iteration Score-CAM computation, the loop periodically yields to the Flutter event loop:

```
// Yield every 5 iterations to keep UI animations running
if (ki % 5 == 0) await Future.delayed(Duration.zero);
```

The `AnalysisResult` returned by the pipeline contains the diagnosis label, confidence score, per-class probability map, a boolean `isNormal` flag, and the file path of the Score-CAM overlay saved to the device's temporary directory.

4.7.5 Local Database Layer

Scan history and user account data are persisted locally using SQLite, accessed through the `sqflite` package. The `DatabaseService` is also a singleton to ensure a single database connection. The schema contains two tables:

```

CREATE TABLE scans (
  id TEXT PRIMARY KEY,
  image_path TEXT NOT NULL,
  body_part TEXT NOT NULL,
  diagnosis TEXT NOT NULL,
  confidence REAL NOT NULL,
  class_probabilities TEXT NOT NULL, -- JSON-encoded map
  timestamp INTEGER NOT NULL,
  notes TEXT,
  heatmap_path TEXT
);

CREATE TABLE users (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password TEXT NOT NULL,
  role TEXT NOT NULL DEFAULT "patient",
  age INTEGER,
  gender TEXT,
  medical_history TEXT,
  created_at INTEGER NOT NULL
);

```

The schema supports database versioning through onUpgrade callbacks, enabling non-destructive schema migrations. The application supports two user roles — Patient and Healthcare Provider — captured during onboarding; Healthcare Providers have access to a specialist dashboard that provides an aggregated view over scan history and patient data.

4.7.6 User Interface and Screen Flow

The application implements a complete, production-ready user interface with both light and dark themes defined centrally in the AppTheme class. Key screens include:

- **Welcome Screen:** Application introduction with animated entry, leading to login or registration.
- **Login and Registration Flow:** Email/password authentication with role selection (Patient or Healthcare Provider) and demographic data collection (age, gender, medical history).
- **Home Dashboard:** Quick-action hub presenting recent scan history, a scan initiation button, and navigation to all secondary screens.
- **Body Part Selection:** An anatomical map enabling the user to specify the affected region before capture, providing additional clinical context stored with each scan record.
- **Capture Screen:** Integrates the device camera or gallery image picker with a preview before submitting to the analysis pipeline.
- **Analyzing Screen:** A step-by-step progress display that updates in real time as each pipeline stage completes (preprocessing → VAE gate → CNN → Score-CAM).

- **Diagnosis Result Screen:** Presents the primary diagnosis, confidence percentage, per-class probability breakdown, and the Score-CAM heatmap overlay. A notes field allows annotation before saving.
- **History Screen:** A chronological list of all saved scans, each showing the diagnosis, body part, confidence score, and timestamp.

4.8 Challenges and Solutions

Several technical challenges arose during implementation. Each was diagnosed systematically and resolved before the final system was deployed.

Challenge	Preprocessing Mismatch Between Training and Inference
Problem	EfficientNetB2 and B3 were trained with an internal Rescaling(1/255) layer embedded in the model graph. During initial TFLite integration, the application was normalizing pixel values to [0, 1] before passing them to the interpreter, resulting in double normalization and near-zero classifier activations.
Resolution	Passing raw [0, 255] pixel values to both EfficientNet TFLite models, while retaining [0, 1] normalization only for the VAE (which was built without an internal rescaling layer). Two separate helper functions — <code>fillFloat32Raw()</code> and <code>fillFloat32()</code> — were implemented for the two normalization modes.

Challenge	VAE Threshold Calibration Across Frameworks
Problem	The VAE was originally calibrated in PyTorch with an MSE anomaly threshold of 0.006. After conversion to TFLite, a systematic offset of approximately +0.002 was observed in reconstruction errors, attributable to differences in floating-point precision and dynamic range quantization.
Resolution	The threshold was recalibrated empirically to 0.008 for the TFLite runtime, producing equivalent anomaly detection behaviour to the original PyTorch model.

Challenge	UI Responsiveness During Score-CAM Computation
Problem	The 30-iteration Score-CAM loop ran synchronously on the main Dart isolate, causing dropped frames and a frozen analyzing screen animation during inference.
Resolution	Inserting <code>await Future.delayed(Duration.zero)</code> every 5 iterations yielded control back to the Flutter event loop, allowing animations to continue without perceptible interruption while the inference executed.

Challenge	Overfitting in MobileNetV2 Phase 3
Problem	The aggressive combination of high dropout (0.5), label smoothing (0.2), and a very low learning rate (1×10^{-6}) in Phase 3 caused under-fitting rather than improved generalization. The model could not converge from the Phase 1 starting point given the combined constraints.
Resolution	Identification of the root cause — restarting from a less-adapted Phase 1 checkpoint rather than continuing from Phase 2's domain-adapted weights. This insight informed the transition to EfficientNet architectures with a cleaner two-phase training protocol and a less aggressive regularization regime.

4.9 Model Performance Summary

The following table summarizes the classification performance achieved across all training phases on the held-out test set of 322 images:

Table 4.10 — Training Phase Summary and Model Progression

Phase	Architecture	Key Technique	Test Accuracy	Best Model
Phase 1	MobileNetV2	Frozen backbone, GAP head	75.16%	No
Phase 2	MobileNetV2	Top 55 layers unfrozen, $LR=1 \times 10^{-6}$	84.16%	MobileNetV2
Phase 3	MobileNetV2	MixUp, label smoothing, cosine LR	73.91% / 76.09% (TTA)	No
Phase 4+	EfficientNetB2	Focal loss, online aug, two-phase	—	EfficientNetB2
Phase 4+	EfficientNetB3	Focal loss, 300×300 , two-phase	—	EfficientNetB3
Final	B2 + B3 Ensemble	Score-CAM, TFLite on-device	—	Deployed

End of Chapter 4 — Implementation